GreenLang Factors FY27

Detailed product-development proposal for the first GreenLang feature: an auditable emissions-factor operating layer for developers, consultants, and climate platforms.

Planning date: 16 April 2026

Founder view: start with Factors now, but build it as a system, not a spreadsheet. The goal for FY27 is a production-ready catalog, API, provenance layer, update engine, and factor-matching workflow that all later GreenLang products can rely on.

Bottom line

- Treat FY27 as the active build year. If your fiscal year runs April–March, GreenLang Factors is already a live Year 1 product, not a future idea.
- A 100,000-factor starting catalog is feasible only if you separate catalog coverage from GA quality. Do not promise 100,000 fully QA-certified public factors on day one.
- Do not hard-code a 'refresh everything every 7 days' rule. Public factor sources update on different cadences: many are annual, some every spring, some ad hoc, and grid-carbon sources can be real-time. Build weekly monitoring plus event-driven publishing instead.
- The right v1 is open-core: public factors plus licensed-source connectors, strong provenance, deterministic calculation logic, and an AI-assisted search-and-mapping layer.

1. Executive recommendation

GreenLang Factors is the correct first product because it is the shared data substrate for GreenLang Comply, SupplierOS, PCF/DPP, and later PlantOS. Without a reliable factor layer, every downstream climate workflow becomes fragile, slow, and consultant-heavy.

My blunt recommendation is to start with a narrower, production-grade definition of Factors:

- Build a factor operating layer, not a static database.
- Use a hybrid architecture: deterministic data normalization and calculation rules, plus AI only for source parsing, mapping assistance, change detection, and search.
- Set three separate targets instead of one vanity number: factors cataloged, factors QA-certified, and factors usable through API in production.
- Separate emissions factors from policy applicability. A regulation change often changes which factor or method is allowed, not the numeric factor itself.

Recommended FY27 product target

- Catalog target: 100,000 normalized factor records across public, preview, and connector-backed datasets.
- Production target: 40,000–60,000 QA-certified records exposed in the default public/enterprise API by end of FY27.
- Update target: 7-day source-watch cycle, <24 hour hotfix process for critical errors, and monthly stable release tags.
- Commercial target: API for developers, enterprise support for consultants/platforms, and 3–8 design partners by year-end.

2. Where GreenLang is in 2026

Assuming an April–March fiscal year, FY27 began on 1 April 2026. That means GreenLang Factors FY27 is already in its first month. Strategically, this is the right time to start: the EU CBAM definitive period began on 1 January 2026, California fixed 10 August 2026 as the first-year SB 253 deadline with Scope 1 and 2 only, and GHG Protocol is actively reviewing parts of the corporate standards stack, including Scope 2. Those shifts increase demand for defensible factor selection, provenance, and fast policy interpretation rather than just raw factor volume.

Signal	Why it matters for Factors	What GreenLang should do
CBAM definitive period active	Importers and advisors need traceable methodology, region, boundary, and source-version metadata for embedded-emissions calculations.	Tag factors by regulatory relevance and keep source/version lineage visible.
SB 253 first-year reporting deadline is Aug 10, 2026	US reporters need fast, auditable Scope 1 and Scope 2 calculations	Prioritize fuel combustion, electricity, and selected mobile-

	now.	combustion categories.
GHG Protocol review is still moving	Method applicability can change before numeric factor tables do.	Keep policy/method rules in a separate versioned layer from the factor store.
Large incumbent datasets already exist	Competing on raw count alone is a losing move.	Compete on trust, speed, API design, provenance, and workflow fit.

3. Product definition

GreenLang Factors should be positioned as the auditable factor layer for climate software. It is not only a database. It is the combination of a normalized factor catalog, factor metadata and provenance, factor search and mapping, update monitoring, versioned releases, enterprise support, and policy-aware applicability logic.

Primary buyers

- Developers embedding carbon calculations into software products or internal tools.
- Sustainability consultants who need a trusted, searchable factor source with citations and release history.
- Climate/accounting platforms that want factor infrastructure without maintaining dozens of source calendars and parsers internally.

Core jobs to be done

- Find the most appropriate factor for a described business activity.
- Understand why that factor was chosen and what assumptions it carries.
- Know which source, release, geography, unit, methodology, and regulatory context the factor belongs to.
- Detect when a source or policy changed and decide whether the factor catalog or only the applicability logic needs updating.
- Serve calculations through an API and preserve reproducibility across time.

What the v1 product is

- A versioned factor catalog with source lineage.
- A public and enterprise API.
- A factor explorer/search interface.
- A factor matching service that maps user activity text to candidate factors.
- A source-watch and release pipeline.
- An enterprise support layer with SLAs and assisted onboarding.

What the v1 product is not

- Not a full LCA suite.
- Not a product-carbon-footprinting workflow end to end.
- Not a compliance report generator by itself.
- Not a guarantee that every licensed/private dataset can be redistributed openly.

4. The product strategy I recommend

The right strategy is open-core, source-aware, and policy-aware. In practice, GreenLang Factors should ship as four layers:

Layer	Purpose	Why it matters
1. Public factor core	Openly redistributable factors from public sources, normalized into GreenLang schema.	This creates adoption, trust, and community usage.
2. Connector layer	Bring-your-own-license or partner-access connectors to licensed/private datasets.	This expands coverage without breaking source licenses.
3. Provenance and release layer	Versioning, citations, change logs, deprecations, validity windows, and reproducibility support.	This is essential for audits and enterprise confidence.
4. Matching and policy layer	Search, semantic mapping, hard filters, ranking, and regulatory/method applicability logic.	This turns a catalog into a usable product.

This structure matters because a number like '100,000 factors' can be misleading. Some factor sources are openly usable, some are licensed, some are annual averages, some are residual mixes, and some are real-time grid signals. GreenLang should not flatten those differences; it should expose them cleanly.

Critical product principle

Never mix together these concepts in one opaque table: (a) the numeric factor itself, (b) the source it came from, (c) the calculation boundary, (d) the accounting method or policy rule that makes it applicable, and (e) the search logic used to retrieve it. Keep them as separate but linked objects.

5. Is 100,000 factors a good starting target?

Yes — but only with the right definition. If 100,000 means 'records in the catalog,' the number is achievable in FY27. If it means 'fully quality-assured, openly redistributable, production-default factors across all categories,' it is too aggressive and will distract the team from shipping a credible v1.

I recommend three coverage labels at launch:

- Certified: QA-complete, production-default, fully documented, stable API exposure.
- Preview: normalized and searchable, but still pending deeper QA or methodology review.
- Connector-only: visible in search metadata or accessible only when a customer has the needed external license/contract.

Recommended FY27 coverage mix

Coverage bucket	Suggested record count	Notes
Certified public	40,000–60,000	Default API result set for most users.
Preview public	20,000–30,000	Searchable, but with lower confidence or narrower documentation coverage.
Connector-backed / licensed	20,000–40,000	Accessible via enterprise connectors or customer-provided licenses.
Total catalog target	100,000+	Use only if coverage labels are explicit in product UX and API.

6. What sources should GreenLang Factors start with?

Start with sources that are authoritative, recurring, and directly useful for Scope 1 and Scope 2, then expand into upstream fuel, transport, waste, spend-based screening, and licensed material/product datasets. The first release slate should optimize for utility and auditability, not for the broadest possible category count.

Source group	Launch stance	Why / caveat
EPA Emission Factors Hub + eGRID	Core public source	High utility for US fuel, electricity, transport, travel, and T&D contexts.
UK DESNZ conversion factors	Core public source	Widely used benchmark set for company reporting and international reporting workflows.
The Climate Registry defaults	Core public source	Helpful for North American reporting defaults and triangulation.
Green-e / residual-mix tables	Core public source for market-based electricity support	Important for unspecified electricity contexts in Scope 2.
GHG Protocol methodology objects	Method/policy metadata, not raw factors	Needed to explain factor selection logic and accounting approach.
Electricity Maps	Connector / premium live grid signal	Important for real-time and forecasted grid-carbon use cases.
IEA emissions factors	Licensed connector	Useful global electricity/heat layer, but licensing must be handled carefully.
ecoinvent / other LCA databases	Licensed connector only	Do not assume open redistribution rights; integrate via license-aware connectors.

Do not anchor the launch on a promise that every desired source will be openly published inside the same database. Some major sources, including ecoinvent and IEA products, operate under license models that require separate rights for redistribution or derived product use.

7. The update model: do not 'refresh everything every 7 days'

A weekly operating rhythm is good. A universal weekly refresh rule is not. Most factor sources are not updated weekly. Some are annual. Some are seasonal. Some are ad hoc. Real-time electricity is a different class entirely. Regulations also change applicability rules more often than they change factor values.

Object	Recommended watch cadence	Recommended publish behavior
Source availability / file change	Daily automated check	Create an ingestion ticket when any source artifact changes.
Source parse + diff	Weekly batch cycle	Run parser, compare to prior version, and classify as major/minor/patch.
Policy / standards watch	Daily monitor + weekly legal review	Update policy objects, notes, and applicability rules separately from numeric factors where possible.
Stable factor release	Monthly	Publish semver release tag with changelog and deprecation notes.
Hotfix release	As needed	Same day for material errors in default results or regulatory mappings.
Real-time grid signal ingestion	Continuous / near real time	Keep outside the static factors release cycle.

Recommended release semantics

- Major release: methodology, boundary, or schema meaning changes.
- Minor release: new factor families, new geographies, new source versions, or materially expanded coverage.
- Patch release: numeric corrections, metadata fixes, broken-link repairs, or policy note corrections.

8. Product architecture

GreenLang Factors should be built as a climate data product with a strong internal object model. The database is only one component. The architecture should have at least these services:

1. Source Registry — master list of all upstream datasets, license class, cadence, parser type, and owner.
2. Ingestion Service — fetches files, pages, API payloads, and documentation snapshots; computes checksums; stores raw artifacts.

3. Parser Service — converts each source into source-specific normalized records; extracts metadata and release information.
4. Canonical Normalizer — maps source-specific records into GreenLang factor schema, unit system, geography model, and gas/GWP model.
5. Quality Engine — validates units, duplicates, outliers, missing lineage, invalid date windows, and contradictory category assignments.
6. Policy and Method Store — keeps accounting-method rules, scope/category applicability, jurisdiction tags, and standards references.
7. Search and Matching Service — supports keyword search, metadata filters, semantic retrieval, reranking, and explainable factor suggestions.
8. Release Manager — creates snapshots, changelogs, deprecations, version tags, and reproducibility manifests.
9. API Gateway and Developer Portal — exposes search, retrieval, metadata, citations, and version pinning.
10. Admin Console — handles approvals, overrides, source monitoring, issue triage, and enterprise support workflows.

Canonical factor schema (minimum)

- factor_id, canonical_name, source_id, source_release, source_record_id
- status (certified / preview / connector), valid_from, valid_to, release_version
- scope, category, activity_family, sector, sub_sector, product_or_material, transport_mode
- geography_type, geography_code, geography_name, language tags
- unit_numerator, unit_denominator, normalization_basis, energy_basis if applicable
- gas_breakdown, co2e_total, gwp_basis, gwp_source, uncertainty_band where available
- methodology_boundary, location_vs_market_based tag, residual_mix tag, upstream_or_direct tag
- license_class, redistribution_rights, citation_text, source_url, checksum, lineage path
- quality_score, validation_flags, deprecation_state, replacement_factor_id where relevant

Make every retrieval reproducible. A user should be able to pin a factor result by GreenLang release version and retrieve the exact source lineage later.

9. The AI/model strategy: use the best architecture, not one magical model

Do not build GreenLang Factors around a single LLM. Build it around a benchmarked multi-stage pipeline. Climate factors are a high-trust data problem. AI is useful in the curation and search layers, but the serving path must remain deterministic and explainable.

Problem	Best approach	Do not do this
Source parsing	Deterministic parsers first; use a reasoning model only where	Do not let an LLM hallucinate factor values from narrative text.

	source structure is messy.	
Activity-to-factor mapping	Hybrid retrieval: hard filters -> semantic retrieval -> reranking -> rule checks -> confidence score.	Do not do one-shot prompt matching with no filters or evaluation set.
Policy change detection	Document diff + reasoning model summarization + human approval.	Do not auto-apply policy changes directly to production logic.
Factor recommendation UX	Show top candidates, rationale, caveats, and source/version metadata.	Do not hide alternative candidates when confidence is low.
Serving path	Deterministic API responses pinned to release versions.	Do not make live production responses depend on nondeterministic prompts.

Recommended model stack

- Rules and ontology layer for scope, geography, unit, boundary, and regulatory compatibility filters.
- Multilingual embedding model for semantic retrieval across activity descriptions, factor names, tags, and documentation snippets.
- Cross-encoder or reranker to improve precision among semantically similar candidates.
- Reasoning model in the back office only: source parsing exception handling, release note drafting, and policy-diff classification.
- Offline evaluation harness with gold-labeled factor-matching cases from real customer activities.

In other words: use frontier AI where it creates leverage, but keep the climate-calculation substrate deterministic.

10. Step-by-step product-development plan

Below is the detailed build sequence I would use if GreenLang started this feature now. The sequence is deliberately disciplined: schema and source rights first, ingestion and QA second, search and API third, and only then production hardening and enterprise support.

Phase 0 — Product framing and operating setup (Weeks 1–2)

- Write the product charter: buyer, job to be done, launch scope, non-scope, and FY27 success metrics.
- Define the factor taxonomy and status model: certified, preview, connector-only, deprecated.
- Create the source-rights policy: what can be redistributed openly, what requires a connector, what requires customer-provided license evidence.
- Stand up repositories, issue templates, schema versioning rules, and release naming convention.
- Appoint explicit owners: product, methodology, data ingestion, QA, and API.

Phase 1 — Source census and licensing guardrails (Weeks 3–5)

- Create a source inventory with cadence, file types, geography, factor classes, license class, and ingestion difficulty.
- Classify every source as public-open, public-usable-with-attribution, restricted, licensed, or partner-only.
- Create a rights matrix that legal can sign off on before engineering publishes anything.
- Prioritize launch sources for Scope 1 and Scope 2 before adding materials/LCA-heavy catalogs.
- Write parser specifications for the first 5–8 sources.

Phase 2 — Canonical schema and ontology (Weeks 6–8)

- Define GreenLang factor schema v0.1 and lock mandatory fields.
- Create unit ontology and conversion engine; include energy, mass, distance, volume, currency, and passenger/freight units.
- Create geography ontology: country, subnational, eGRID subregion, grid zone, route corridor, and supplier region where relevant.
- Create methodology ontology: direct vs upstream, location-based vs market-based, residual mix, spend-based vs activity-based, GWP basis.
- Create deprecation model and replacement pointers.

Phase 3 — Raw ingestion framework (Weeks 9–11)

- Build source fetchers for web pages, XLSX, CSV, API endpoints, and documentation PDFs.
- Store all raw source artifacts in immutable object storage with checksums and retrieval timestamps.
- Create parser harnesses with source-specific tests and failure alerts.

- Implement ingestion metadata tables: source version detected, parse status, row counts, anomalies, and ownership.

Phase 4 — Normalization and lineage (Weeks 12–14)

- Map raw records into canonical factor objects.
- Normalize units and retain original raw values and units in lineage fields.
- Attach release dates, validity windows, source URLs, and citation text.
- Compute duplicate fingerprints and candidate-duplicate clusters.
- Persist lineage links so every canonical factor can be traced back to raw source rows/files.

Phase 5 — Quality engine and methodology review (Weeks 15–18)

- Build validation checks for missing fields, invalid units, zero or negative outliers where impossible, impossible date windows, and broken geography codes.
- Create a methodology review workflow for factors needing human approval.
- Build QA dashboards: counts by source, counts by status, failing validation rules, and drift vs previous releases.
- Create the first audit package template for a factor: raw evidence, parser logs, normalized record, QA result, and reviewer decision.

Phase 6 — Search, retrieval, and factor matching (Weeks 19–22)

- Build faceted search on scope, category, geography, year, unit, methodology, and source.
- Build semantic index over factor names, descriptions, tags, and source notes.
- Build candidate reranking and confidence scoring.
- Create gold-label evaluation set from 300–500 real-world activity descriptions across initial categories.
- Add explanations: why this factor is suggested, what assumptions it implies, and what alternatives exist.

Phase 7 — API, developer portal, and version pinning (Weeks 23–26)

- Ship read APIs: search factors, get factor by ID, list releases, compare releases, retrieve lineage, retrieve deprecation chain.
- Ship write/admin APIs for source monitoring, QA overrides, and issue management.
- Add version pinning so users can reproduce results across time.
- Write developer docs with code samples, factor-selection guidance, and source limitations.
- Add usage logging and API key management.

Phase 8 — Update engine and policy watch (Weeks 27–30)

- Create automated watch jobs for official source pages, downloads, and documentation changes.
- Create document diff jobs for standards and regulations that can affect applicability logic.

- Create change-classification workflow: numeric factor change, metadata change, policy change, parser breakage, documentation-only change.
- Generate release notes automatically, but require human approval before publishing.

Phase 9 — Design-partner pilots (Weeks 31–38)

- Run pilots with 3–5 partner types: a consultant, a climate SaaS platform, an internal enterprise sustainability team, and optionally a developer tooling partner.
- Collect real activity descriptions and failed searches to improve matching precision.
- Measure time-to-factor, mismatch rate, support tickets, and citation completeness.
- Decide which preview categories should be promoted to certified status.

Phase 10 — GA and enterprise hardening (Weeks 39–52)

- Add SLA monitoring, enterprise support runbooks, outage communication templates, and backup/restore tests.
- Ship audit exports, changelog exports, and source-diff views for enterprise users.
- Create pricing tiers and contract terms.
- Launch public docs, docs examples, and community contribution guidelines.
- Publish v1 GA release and 90-day post-launch roadmap.

11. FY27 quarter-by-quarter roadmap

Quarter	Primary outcome	Must-have deliverables
Q1 FY27 (Apr–Jun 2026)	Foundation	Product charter, source-rights matrix, schema v0.1, first public-source parsers, raw artifact store.
Q2 FY27 (Jul–Sep 2026)	Usable alpha	Normalized catalog, QA engine, first search UX, initial API, first stable monthly release.
Q3 FY27 (Oct–Dec 2026)	Pilot beta	Semantic matching, policy-watch workflow, enterprise pilot package, 3–5 design partners live.
Q4 FY27 (Jan–Mar 2027)	GA	Version-pinned API, certified catalog subset, changelog system, enterprise support process, public launch.

12. Team required to build this well

You do not need a huge team for v1, but you do need the right shape. A founder-only or pure-ML team will struggle because the hard part is data rights, methodology, and product discipline.

Role	Why required in FY27	When to hire / assign
Product lead / founder owner	Owns scope discipline, design partner feedback, and release decisions.	Now
Climate methodology lead	Owns factor policy, standards interpretation, QA review, and trust layer.	Now
Data platform engineer	Builds ingestion, normalization, lineage, and release pipeline.	Now
Backend/API engineer	Builds serving layer, auth, version pinning, and docs samples.	Now or next 30 days
Search/ML engineer	Builds semantic retrieval, reranking, and evaluation harness.	By Phase 6
QA/data analyst	Runs validation workflows and release readiness checks.	By Phase 5 or contractor
DevRel / technical writer	Creates docs, examples, SDK content, and contribution flows.	Late FY27 / contractor
Part-time legal/commercial advisor	Cleans up data-rights and enterprise support terms.	Now, fractional

13. Product KPIs that actually matter

KPI	Target by FY27 end	Why it matters
Catalog coverage	100,000+ total records	Shows source breadth, but only useful with status labels.
Certified coverage	40,000–60,000	Shows production-grade utility.
API freshness SLA	Stable monthly releases + hotfixes in <24h for critical errors	Builds enterprise trust.
Search success	Top-5 candidate recall >95% on gold eval set	Measures whether users can find usable factors.
Selection precision	Top-1 precision >80% in initial categories	Measures mapping quality.
Citation completeness	100% for certified factors	Mandatory for auditability.
Lineage completeness	100% raw-to-canonical trace for certified factors	Mandatory for defensibility.
Pilot conversion	At least 3 paying or committed design partners	Proves commercial fit.
Time to onboard a new public source	<10 working days after parser spec exists	Shows operating leverage.

14. Risks and what to do about them

Risk	Why it is real	Mitigation
Overpromising catalog size	Raw count can hide poor quality or licensing risk.	Separate total catalog from certified catalog and connector coverage.
Licensing mistakes	Some valuable datasets cannot be freely redistributed.	Use a signed rights matrix before publication and default to connector mode.
Methodology drift	Standards and policy rules continue to evolve.	Keep policy logic versioned separately; maintain legal/methodology review cadence.
LLM overreach	Model hallucinations can corrupt factor selection.	Keep production serving deterministic; AI only augments, never invents values.
Too many categories too early	Breadth kills shipping speed.	Start with Scope 1 + Scope 2 + selected transport/upstream categories.
Weak documentation	A factor API without trust artifacts will not win enterprise users.	Ship provenance, changelog, and examples alongside the API.

15. The first 30 / 60 / 90 days

First 30 days

- Freeze product scope for v1.
- Write schema v0.1 and rights matrix.
- Set up source registry and raw artifact storage.
- Build first parsers for EPA, DESNZ, and one electricity-specific source.
- Define release semantics and issue taxonomy.

Days 31–60

- Ship canonical normalization and first certified factors set.
- Stand up QA dashboards and validator library.
- Create the first factor explorer internal UI.
- Start assembling gold-labeled factor-matching evaluation set.
- Run first design-partner reviews on data fields, not on UI polish.

Days 61–90

- Release internal alpha API.
- Add semantic retrieval and candidate ranking.
- Stand up weekly source-watch jobs and diff workflow.
- Create changelog and deprecation pipeline.
- Prepare the first enterprise pilot package.

16. Detailed workstream checklist

The following checklist is designed to be execution-ready. It can be imported into a project tracker with minimal changes.

Product and methodology

ID	Task
P1	Write the PRD and launch scope.
P2	Define user personas and 20 core use cases.
P3	Define factor status model and release policy.
P4	Define methodology review checklist.
P5	Define deprecation and replacement policy.
P6	Define support boundaries for public vs enterprise users.

Source rights and governance

ID	Task
G1	Inventory every upstream dataset and page.
G2	Classify redistribution rights.
G3	Record required attribution text.
G4	Record release cadence and watch mechanism.
G5	Create approval gate before any new source goes public.
G6	Create exception path for connector-only sources.

Data engineering

ID	Task
D1	Create raw artifact store with checksums.
D2	Create parser framework and test harness.
D3	Create normalization mappings and unit conversions.
D4	Create lineage store and evidence links.
D5	Create duplicate detection and merge policy.
D6	Create release snapshot and rollback tooling.

Quality and trust

ID	Task

Q1	Create schema validator library.
Q2	Create unit sanity checks and outlier rules.
Q3	Create methodology review queue.
Q4	Create certified-vs-preview promotion criteria.
Q5	Create audit export template.
Q6	Create release signoff checklist.

Search and matching

ID	Task
M1	Define retrieval schema and searchable fields.
M2	Build keyword/facet search.
M3	Build semantic index.
M4	Build reranker and confidence scoring.
M5	Build evaluation set and scorecards.
M6	Expose explanation traces in product UX.

API and developer experience

ID	Task
A1	Design factor search API.
A2	Design release/version API.
A3	Design lineage and citations API.
A4	Ship auth, API keys, and rate limits.
A5	Write code examples and quickstarts.
A6	Create SDK backlog only after core API stabilizes.

Update and policy watch

ID	Task
U1	Create source-watch jobs.
U2	Create document diff engine.
U3	Create policy-change classification workflow.
U4	Route policy changes to methodology review.
U5	Create changelog generator.
U6	Create hotfix release procedure.

Commercial readiness

ID	Task
C1	Define free/dev/enterprise packaging.
C2	Write enterprise support policy and SLAs.
C3	Define pilot onboarding workflow.
C4	Create security and procurement FAQ.
C5	Create API usage reporting for enterprise billing.
C6	Create customer issue severity matrix.

17. Recommended launch scope for FY27

If you want the fastest path to value, launch with a disciplined factor set instead of trying to cover every climate category immediately.

Include in FY27 GA	Reason
Stationary fuel combustion	Immediate Scope 1 utility and broad enterprise demand.
Mobile combustion	Needed for fleets, company vehicles, and selected logistics reporting.
Electricity location-based factors	Core Scope 2 requirement.
Electricity residual-mix / market-based support	Needed for more mature Scope 2 accounting workflows.
Selected upstream fuel and T&D factors	Adds practical completeness without excessive complexity.
Business travel and product transport starter categories	Useful to consultants and platforms early.
Waste and water starter categories	Good utility with manageable complexity.
Factor matching + provenance + release history	This is the product moat, not an add-on.

Defer beyond FY27 or keep connector-only	Reason
Deep LCA material catalogs	Licensing and methodology complexity can consume the roadmap.
Full product-footprint workflow	Should live in PCF/DPP product later.
Carbon-credit MRV factors	Different trust and evidence model.
Sector-specific industrial process models	Belong in later GreenLang modules, not Factors v1.
Custom consulting logic embedded into the public core	Creates maintenance debt and product confusion.

18. Final judgment

Start GreenLang with Factors. It is the right first wedge. But build it as a climate data operating layer with provenance, source rights, policy logic, and deterministic serving. If you do that, the rest of the GreenLang stack becomes easier. If you reduce it to a giant spreadsheet of 100,000 rows updated every Friday, it will break under its own weight.

My strongest recommendation is this: ship trust before you ship breadth. The category does not need another big factor list; it needs a reliable factor system.

Appendix A — Selected external references used in planning

These references were used to calibrate regulatory timing, factor-source cadence, licensing constraints, and the competitive baseline. They are included for planning and verification.

R1. US EPA GHG Emission Factors Hub; page states the 2025 annual update included purchased electricity from eGRID, transport, travel, and T&D-loss related updates. — https://www.epa.gov/climateleadership/ghg-emission-factors-hub

R2. US EPA eGRID page; notes latest update to 2023 data. — https://www.epa.gov/egrid

R3. UK DESNZ greenhouse gas reporting conversion factors 2025; official annual publication. — https://www.gov.uk/government/publications/greenhouse-gas-reporting-conversion-factors-2025

R4. The Climate Registry General Reporting Protocol; states TCR compiles default emission factors from public sources and updates them annually in CRIS. — https://theclimateregistry.org/wp-content/uploads/2025/06/General-Reporting-Protocol-3.0_The-Climate-Registry_rev.6-5-25.pdf

R5. Center for Resource Solutions residual mix rate tables; says Green-e residual mix emissions rates are published every spring. — https://resource-solutions.org/residual-mix/

R6. 2025 Green-e residual mix release page; released 29 January 2026. — https://resource-solutions.org/2025-residual-mix/

R7. Electricity Maps platform page; states data is available historically, in real time, and forecasted, with coverage of 190+ countries. — https://www.electricitymaps.com/

R8. GHG Protocol public consultations page; states consultations on Scope 2 and consequential electricity accounting ran to 31 January 2026. — https://ghgprotocol.org/ghg-protocol-public-consultations

R9. EU Access2Markets note on start of CBAM definitive period from 1 January 2026. — https://trade.ec.europa.eu/access-to-markets/en/news/start-definitive-period-cbam-eu

R10. CARB note on Aug. 10, 2026 first-year SB 253 reporting deadline and Scope 1/2-only first year. — https://ww2.arb.ca.gov/news/carb-approves-climate-transparency-regulation-entities-doing-business-california

R11. Climatiq homepage showing 944k scientifically vetted emission factors and 300+ data sources. — https://www.climatiq.io/

R12. ecoinvent license page; annual licenses and access through ecoQuery. — https://ecoinvent.org/licenses/

R13. IEA Emissions Factors 2025 product page; annual electricity/heat factors with licensing restrictions and yearly September release schedule. — https://www.iea.org/data-and-statistics/data-product/emissions-factors-2025

R14. India PIB note referencing nearly 490 obligated entities across seven energy-intensive sectors under CCTS. — https://www.pib.gov.in/PressReleasePage.aspx?PRID=2243377&reg=3&lang=2